UNIVERSITY OF ESWATINI

PRODUCER-CONSUMER PROBLEM IMPLEMENTATION

CSC411 - Integrative Programming Technologies Mini Project

Submitted by:

Neliswa Maziya - 202203810

Ayanda Mhlanga - 202200841

Submitted to:
Dr. Nxumalo

Submission Date: November, 2025

Repository link: https://github.com/maziyaneliswanhlanhla/CSC-411_PROJECT

Contents

ABSTRACT

This report presents an implementation of the Producer-Consumer synchronization problem using PHP. The project demonstrates concurrent programming concepts including process synchronization, mutual exclusion, and bounded buffer management. The system generates random student data, serializes it to XML format, and processes it while maintaining proper synchronization between producer and consumer processes. Three implementations are provided: file-based concurrent execution, sequential demonstration, and network-based socket communication.

1. INTRODUCTION

1.1 Project Overview

The Producer-Consumer Problem is a classic example of a multi-process synchronization problem. This project implements a solution where the Producer generates student information and stores it in XML format, the Consumer retrieves and processes the student data, a Bounded Buffer acts as shared memory space with limited capacity, and Synchronization mechanisms ensure data integrity and prevent race conditions.

1.2 Objectives

The primary objectives of this project are to implement the Producer-Consumer problem with proper synchronization, demonstrate XML data serialization and deserialization, develop a thread-safe bounded buffer with capacity constraints, implement network-based communication using socket programming, utilize version control systems for collaborative development, and create comprehensive documentation and demonstration materials.

2. BACKGROUND AND THEORY

2.1 The Producer-Consumer Problem

The Producer-Consumer problem involves two types of processes. Producers create data items and place them in a buffer. Consumers retrieve data items from the buffer and process them. The challenge is to coordinate these processes to prevent buffer overflow where the producer adds data to a full buffer, buffer underflow where the consumer removes data from an empty buffer, race conditions where multiple processes access the buffer simultaneously, and data corruption where there is an inconsistent buffer state due to concurrent access.

2.2 Synchronization Requirements

Three fundamental rules must be enforced. First, the producer must not add items when the buffer is full (producer blocking). Second, the consumer must not remove items when the buffer is empty (consumer blocking). Third, only one process can access the buffer at any time (mutual exclusion).

2.3 Classical Solutions

Traditional solutions employ semaphores including counting and binary semaphores for synchronization, monitors as high-level synchronization constructs, and mutexes and condition variables as low-level locking mechanisms. Our implementation uses file-based locking as PHP's native mechanism for inter-process synchronization.

2.4 XML Data Format

XML (eXtensible Markup Language) provides platform-independent data representation, human-readable structure, built-in validation capabilities, and wide tool and library support.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 System Architecture

The system consists of a Producer Process that produces data and sends it to a Shared Buffer with a maximum size of 10 elements. The Consumer Process receives data from the Shared Buffer. The Producer creates XML files numbered 1 through 10 in a shared directory. The buffer maintains its state in buffer.json and buffer.json.lock files. The Consumer displays output after processing the data.

3.2 Component Design

The ITStudent Class is responsible for storing student information including name, ID, programme, courses, and marks. It serializes data to XML format and deserializes from XML format. It also calculates average marks, determines pass/fail status, and displays formatted output. The class follows object-oriented principles with encapsulation, providing getters and setters for all properties. XML serialization is handled internally, abstracting the complexity from external code.

The SharedBuffer Class maintains a bounded queue with a maximum of 10 elements. It provides thread-safe produce and consume operations, enforces buffer capacity constraints, and persists state across processes. It uses file-based storage and locking to enable inter-process communication. The JSON format allows easy serialization of the queue structure.

The Producer Class generates random student data and creates ITStudent objects. It serializes data to XML and stores it in a shared directory, then adds references to the buffer. The class implements randomization algorithms for realistic data generation. Its modular design allows easy extension of data generation logic.

The Consumer Class monitors the buffer for available items, reads XML files, and deserializes them to ITStudent objects. It processes and displays data, cleans up processed files, and removes buffer references. The design separates concerns by handling file I/O, deserialization, and display independently.

3.3 Data Flow

The Producer Process follows these steps: it generates a random ITStudent, serializes it to XML, checks if the buffer is full and waits if yes, saves the XML to shared/studentN.xml, adds N to the buffer, and repeats the process.

The Consumer Process follows these steps: it checks if the buffer is empty and waits if yes, removes N from the buffer, reads shared/studentN.xml, deserializes XML to ITStudent, calculates average and status, displays information, deletes the XML file, and repeats the process.

4. IMPLEMENTATION DETAILS

4.1 Random Data Generation

The producer generates realistic Eswatini student data. Names are a combination of common Swazi first names such as Thabo, Sipho, and Nomsa, and surnames such as Dlamini, Nkosi, and Simelane. Student IDs are 8-digit numbers generated using the sprintf function with the format '%08d' and random numbers between 10000000 and 99999999.

Programmes include BSc Computer Science, BSc Information Technology, BSc Software Engineering, and BSc Data Science. Courses are a random selection of 4 to 6 courses from a predefined list. Marks are random integers between 35 and 100.

4.2 XML Serialization

The XML structure begins with an XML version declaration followed by a student element. Inside the student element, there is a name element, a studentID element, a programme element, and a courses element. Within the courses element, there are multiple course elements, each containing a name element and a mark element.

The implementation uses PHP's SimpleXMLElement for creation. HTML special characters are escaped using htmlspecialchars() to prevent XML injection.

4.3 Synchronization Mechanism

The file-based locking mechanism uses the acquireLock function to open a lock file and apply an exclusive lock using the flock function with LOCK_EX parameter. If the lock is successfully acquired, it returns the file pointer, otherwise it returns false.

The locking has the following characteristics: it uses exclusive locks (LOCK_EX), operates at the process level, is blocking (processes wait for lock availability), and fairness is OS-dependent but generally follows FIFO order.

All buffer operations including produce, consume, isEmpty, and isFull are protected by locks, ensuring atomic operations.

4.4 Buffer Management

The buffer is stored in JSON format as an array of integers, for example [1, 2, 3, 4, 5].

The produce operation follows these steps: acquire lock, load buffer from file, check if full (count greater than or equal to 10), if not full then append item, save buffer to file, and release lock.

The consume operation follows these steps: acquire lock, load buffer from file, check if empty (count equals 0), if not empty then shift first item, save buffer to file, and release lock.

4.5 Average Calculation and Pass/Fail Logic

The calculateAverage function checks if the courses array is empty and returns 0 if true. Otherwise, it calculates the total using array_sum on the courses and divides by the count of courses.

The getPassFailStatus function calculates the average and returns "PASS" if the average is greater than or equal to 50, otherwise it returns "FAIL".

The system uses a 50% threshold as specified in the requirements.

5. TESTING AND RESULTS

5.1 Test Scenarios

Test 1 was a basic functionality test. The objective was to verify core produce and consume operations. The steps were to clear all previous data, produce 5 students, consume 5 students, and verify all files were processed. The result was that all students were successfully produced and consumed.

Test 2 was a buffer capacity test. The objective was to verify the buffer size constraint. The steps were to produce 15 students rapidly, monitor buffer size, and verify the producer waits when the buffer reaches 10. The result was that the buffer never exceeded 10 elements and the producer correctly waited when full.

Test 3 was a synchronization test. The objective was to verify no race conditions occur. The steps were to run producer and consumer concurrently, process 20 students, and check for any file access errors or data corruption. The result was that no race conditions were detected and all 20 students were processed correctly.

Test 4 was an empty buffer handling test. The objective was to verify the consumer waits when the buffer is empty. The steps were to start the consumer first, start the producer after 5 seconds, and verify the consumer waits. The result was that the consumer correctly waited until data was available.

Test 5 was an XML integrity test. The objective was to verify data preservation through serialization. The steps were to generate a student with known data, serialize to XML, deserialize back, and compare all fields. The result was that all data was correctly preserved with no information loss.

5.2 Performance Metrics

The average execution time was approximately 1 second per student for both the producer and the consumer. The total time for 10 students in concurrent mode was 12 to 15 seconds. Memory usage averaged 5 to 10 MB with a peak of approximately 15 MB when the buffer was full. The file system impact included 10 XML files created with an average size of 1 to 2 KB each, 1 buffer.json file of approximately 100 bytes, and 1 buffer.json.lock file of 0 bytes.

5.3 Sample Output

The student information display shows the student's name, student ID, and programme. It lists courses and marks in a formatted table. The display includes the average mark calculated to two decimal places and the status showing either PASS or FAIL.

An example output shows: Name is Nomsa Simelane, Student ID is 45678912, Programme is BSc Information Technology. The courses are CSC411 Integrative Programming with 78%, CSC301 Data Structures with 65%, CSC402 Web Development with 82%, and CSC403 Mobile Computing with 71%. The Average Mark is 74.00% and the Status is PASS.

6. SOCKET PROGRAMMING IMPLEMENTATION

6.1 Architecture

The socket implementation follows a client-server model. The Server acts as the Producer and listens on a specified port (default is 8888), accepts a single client connection, sends the student count, sends XML data with delimiters, and waits for acknowledgments.The Client acts as the Consumer and connects to the server, receives the student count, receives and processes XML data, sends acknowledgments, and closes when done.

6.2 Communication Protocol

The protocol works as follows: the Client connects to the Server, the Server sends the Student Count, then for each student the Server sends XML data with a delimiter "|||END|||", the Client processes the data and sends an "ACK" acknowledgment. After all students are processed, the Server sends a "DONE" signal and both close their connections.

6.3 Implementation Details

Socket creation uses the socket_create function with AF_INET for IPv4 protocol, SOCK_STREAM for TCP connection, and SOL_TCP for TCP protocol. Data transmission uses a custom delimiter "|||END|||" to ensure complete XML documents are transmitted without fragmentation issues. Error handling includes comprehensive error checking for socket creation failures, bind failures, connection failures, and transmission errors.

6.4 Advantages over File-Based Approach

Network distribution allows the producer and consumer to run on different machines. Real-time communication eliminates file system overhead. The system is scalable and can support multiple consumers with minor modifications. It has industry relevance as it mirrors real-world distributed systems.

7. VERSION CONTROL AND COLLABORATION

7.1 GitHub Repository Structure

The repository is organized with a src directory containing ITStudent.php, SharedBuffer.php, producer.php, consumer.php, main.php, socket_server.php, and socket_client.php. There is a docs directory with README.md and ProjectReport.pdf. A tests directory contains test_scenarios.txt. A demo-video directory contains presentation.mp4. The root contains .gitignore and LICENSE files.

8. CHALLENGES AND SOLUTIONS

8.1 Challenge 1: Inter-Process Synchronization

The problem was that PHP doesn't have built-in threading support like Java or Python. The solution was to implement file-based locking using flock(). While not as elegant as semaphores, it provides reliable mutual exclusion across separate PHP processes. The lesson learned was that understanding the tools available in your language ecosystem is crucial for effective problem-solving.

8.2 Challenge 2: XML Data Integrity

The problem was that special characters in names were breaking the XML structure. The solution was to use htmlspecialchars() for encoding and proper decoding on deserialization. The lesson learned was to always sanitize data before serialization to prevent injection attacks and parsing errors.

8.3 Challenge 3: Socket Buffer Management

The problem was that TCP streams don't preserve message boundaries, causing incomplete XML transmission. The solution was to implement a custom delimiter "|||END|||" to mark message boundaries and buffering logic to reassemble complete messages. The lesson learned was that network programming requires careful protocol design to handle data fragmentation.

8.4 Challenge 4: Deadlock Prevention

The problem was the risk of deadlock if a lock is not released properly. The solution was to ensure every lock acquisition is followed by release in finally-equivalent logic. PHP's file handle closure automatically releases locks. The lesson learned was that resource management is critical in concurrent systems.

9. CONCLUSION

9.1 Project Outcomes

This project successfully demonstrates the Producer-Consumer problem implementation with comprehensive implementation of three working versions (file-based, sequential, socket-based), proper synchronization where all buffer rules are enforced without race conditions, XML processing with robust serialization and deserialization, network communication through functional socket-based distribution, and professional development including version control, documentation, and collaboration.

9.2 Learning Outcomes

Through this project, we gained practical experience in concurrent programming by understanding synchronization challenges and solutions, inter-process communication using file-based and network-based approaches, data serialization through XML processing and validation, software engineering practices including version control, documentation, and testing, and problem-solving by debugging synchronization issues and platform compatibility.

9.3 Future Enhancements

Potential improvements include support for multiple producers and consumers by scaling to N producers and M consumers, implementation of a priority queue to process urgent students first, use of persistent storage through a database instead of files, development of a web interface for browser-based monitoring and control, implementation of load balancing to distribute work across multiple consumers, and addition of error recovery to handle process crashes gracefully.

10. REFERENCES

1. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
2. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
3. PHP Manual. (2024). PHP: Filesystem Functions. Retrieved from https://www.php.net/manual/en/ref.filesystem.php
4. PHP Manual. (2024). PHP: Sockets. Retrieved from https://www.php.net/manual/en/book.sockets.php
5. W3C. (2024). Extensible Markup Language (XML). Retrieved from https://www.w3.org/XML/
6. Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). UNIX Network Programming, Volume 1: The Sockets Networking API (3rd ed.). Addison-Wesley.
7. Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.
8. Dijkstra, E. W. (1965). Cooperating sequential processes. Technical Report EWD-123, Technological University, Eindhoven, Netherlands.